



New Relic

for Junior DevOps Engineers

From Zero to Production Monitoring



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

What Is New Relic and Why Do DevOps Engineers Use It?

What Is New Relic?

New Relic is a cloud-based **observability platform** that helps you monitor, understand, and troubleshoot your applications, infrastructure, and services in real time.

Why Do Engineers Use It?

- ✓ See how applications and infrastructure are performing.
- ✓ Find and fix problems faster.
- ✓ Understand the full user experience.
- ✓ Monitor cloud, containers, and services in one place.
- ✓ Use data to make better decisions.

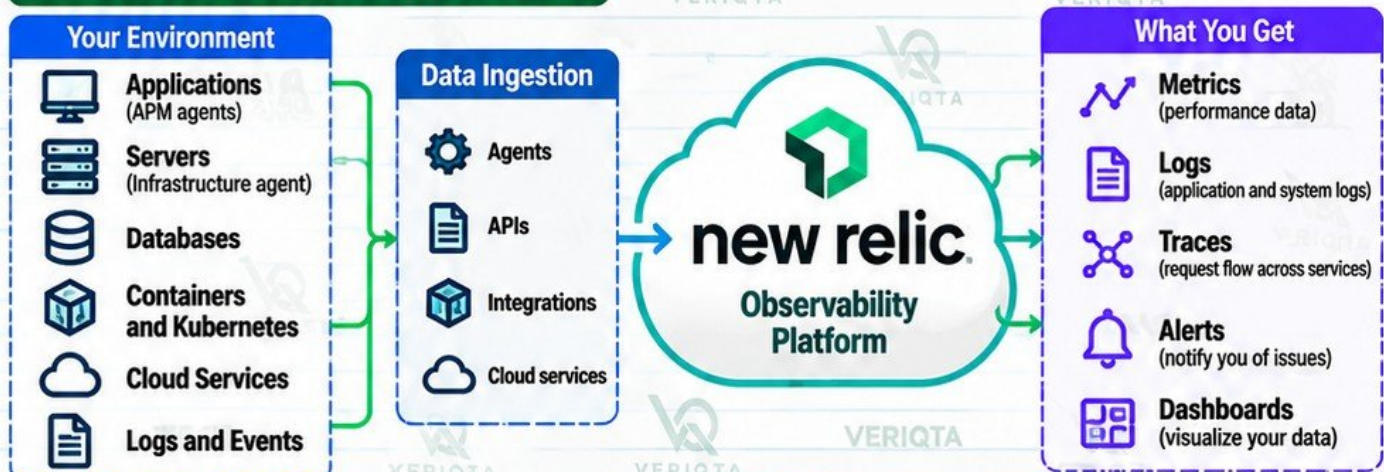
The Problem It Solves

- ❗ Applications are slow but you don't know why.
- ❗ Services keep failing and there is no clear evidence.
- ❗ Logs, metrics, and traces are in different tools.
- ❗ It is hard to see the complete view of your system.
- ❗ Engineers spend too much time guessing the cause of issues.

Key Features

- ✓ Application Performance Monitoring (APM)
- ✓ Infrastructure Monitoring
- ✓ Log Management
- ✓ Distributed Tracing
- ✓ Dashboards and Visualization
- ✓ Alerting and Incident Management
- ✓ Real User Monitoring (RUM)
- ✓ Support for cloud, containers and Kubernetes

How It Works (Simple View)



Junior Engineer Insight

New Relic does not just tell you that something is down. It gives you the data to understand why it is happening and helps you fix it.



Real DevOps Use Cases

- ✓ Investigate slow API responses
- ✓ Find the root cause of application errors
- ✓ Monitor server and container health
- ✓ Track user experience and uptime
- ✓ Set up alerts before users are affected



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Monitoring vs. Observability: Metrics, Logs, Traces, and Events

What Is Monitoring?

Monitoring tells you **if your systems are working as expected**. It tracks known metrics and alerts you when something crosses a threshold (e.g., high CPU, high error rate).



- ✓ Tracks specific metrics
- ✓ Answer: Is it working?
- ✓ Uses dashboards and alerts
- ✓ Good for known problems

What Is Observability?

Observability gives you a **complete understanding** of what is happening inside your systems. It uses metrics, logs, traces, and events to help you **investigate unknown problems** and find the root cause.



- ✓ Uses all telemetry data
- ✓ Answer: Why is it happening?
- ✓ Helps you investigate and troubleshoot
- ✓ Essential for complex systems (microservices, cloud, containers)

The Four Types of Telemetry in New Relic

Metrics



Numeric data collected over time.

Examples:

- CPU usage
- Memory usage
- Response time
- Request count
- Error rate

Used to understand performance and trends.

Logs



Detailed text records of what happened.

Examples:

- Application logs
- System logs
- Error messages
- Access logs
- Container logs

Used to find details and error causes.

Traces



Show the journey of a single request across multiple services.

Examples:

- Request flow
- Service calls
- Time taken in each service
- Identify slow components
- Find where a request fails

Used to find latency and root cause.

Events



Discrete occurrences that happen in your system.

Examples:

- Deployments
- Config changes
- Service restarts
- Scaling events
- Custom application events

Used to understand changes and their impact.

How They Work Together

User Request

Application and Services

Metrics
Logs
Traces
Events

All telemetry is collected and correlated in New Relic. Engineers use metrics, logs, traces, and events together to get the full picture, investigate issues, and find the root cause faster.

new relic.
Observability Platform



Real Engineering Example

A user reports the application is slow.

- 1 Metrics show high response time.
- 2 Traces show which service is slow.
- 3 Logs show error messages.
- 4 Events show a recent deployment.

Together, you can identify the exact cause, instead of guessing.



Key Takeaway

Monitoring tells you something is wrong. Observability helps you understand **why** and **fix** it.



Junior Engineer Tip

In real DevOps work, you always combine metrics, logs, traces, and events to investigate and resolve issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

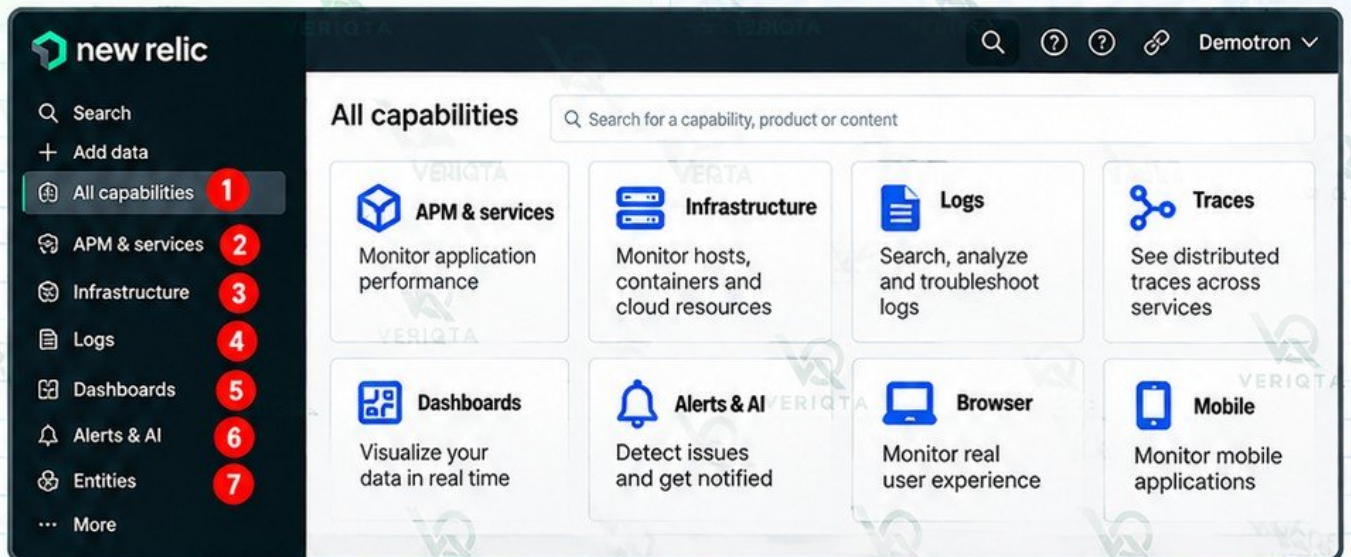
Understanding the New Relic Platform and Interface

What Is the New Relic Interface?

The New Relic interface is a **web-based console** where you can view all your telemetry data, create dashboards, set up alerts, and investigate issues across your applications, infrastructure, and services.

Key Areas in the Interface

- 1 All Capabilities – access all features
- 2 APM & Services – monitor applications
- 3 Infrastructure – monitor servers and containers
- 4 Logs – search and analyze logs
- 5 Dashboards – visualize your data
- 6 Alerts & AI – set alerts and detect issues
- 7 Entities – view all monitored resources



Entities



- ✓ Represent all monitored resources.
- ✓ Includes applications, hosts, containers, databases and more.
- ✓ Gives you a unified view of your environment.

APM & Services



- ✓ Monitors application performance.
- ✓ Shows response time, throughput and error rate.
- ✓ Helps you find slow endpoints and failing services.

Infrastructure



- ✓ Monitors servers, containers and cloud resources.
- ✓ Tracks CPU, memory, disk, network and more.
- ✓ Helps you detect infrastructure issues before users are affected.

Logs



- ✓ Collects and stores logs from your systems.
- ✓ Allows you to search, filter and analyze logs.
- ✓ Helps you understand errors and system behaviour.

Dashboards



- ✓ Visualize metrics, logs and traces in one place.
- ✓ Create custom dashboards for your team.
- ✓ Helps you monitor key service health and business metrics.

Alerts & AI



- ✓ Create alert conditions and notification channels.
- ✓ Get notified when something is wrong.
- ✓ Uses AI to help detect anomalies and issues early.



Junior Engineer Tip

Start by exploring the All Capabilities menu. This helps you understand what New Relic can do and where to find each feature.



Real World Use

In a production environment, engineers use the New Relic interface to monitor services, investigate incidents, and create dashboards for the team.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Creating a New Relic Account and Understanding Data Ingestion

1 Create a New Relic Account

- 1 Go to <https://newrelic.com>
- 2 Click **Sign Up**.
- 3 Create your account (you can start with the free plan).
- 4 Verify your email address.
- 5 Log in to your New Relic account.



Create your free account
Start monitoring in minutes.

Create account

2 Understand Data Ingestion

New Relic receives telemetry data from your applications, infrastructure and services. This process is called **data ingestion**.

Telemetry data includes:



Metrics
(numeric data)



Logs
(text records)



Traces
(request flow)



Events
(discrete occurrences)

3 How Data Ingestion Works

Your Environment

- Servers**
(Linux/Windows)
- Applications**
(APM agents)
- Containers**
(Docker/Kubernetes)
- Cloud Services**
(AWS, Azure, GCP)
- Logs and Events**
(from apps and systems)

New Relic Ingestion



New Relic Platform

- Visualize data in dashboards
- Search and analyze logs (NRQL)
- View traces and service maps
- Set alerts
- Investigate and troubleshoot issues

4 API Keys in New Relic

API keys allow your servers, applications and tools to send data to New Relic securely.

Common types of keys:



Ingestion Key

Used by agents to send telemetry data (metrics, logs, traces, events).



User API Key

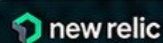
Used for querying data (NRQL) and automation.



License Key (for some agents)

Used to connect agents to your account.

5 Where to Find Your Ingestion Key



- Search
- + Add data
- All capabilities
- APM & services
- Infrastructure
- Logs
- Dashboards
- Alerts & AI
- Integrations
- API keys**

API keys

Ingestion keys User keys License keys

Ingestion keys

Use this key to send data to New Relic from your apps, servers and infrastructure.

Create ingestion key

6 Why Ingestion Costs Matter

New Relic pricing is based on the amount of data you ingest and store. More data means higher costs.

Factors that increase cost:

- ↑ Large amount of logs
- ↑ High metric volume (many hosts/containers)
- ↑ High trace volume (busy applications)
- ↑ Long data retention period
- ↑ Multiple environments (dev, staging, production)



Junior Engineer Tip

Start with the free plan to learn. Understand what data you are sending to avoid unexpected costs later.



Best Practice

- ✓ Use ingestion keys securely (do not share publicly).
- ✓ Monitor your data usage.
- ✓ Ingest only the data you need.
- ✓ Set retention based on your requirements.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

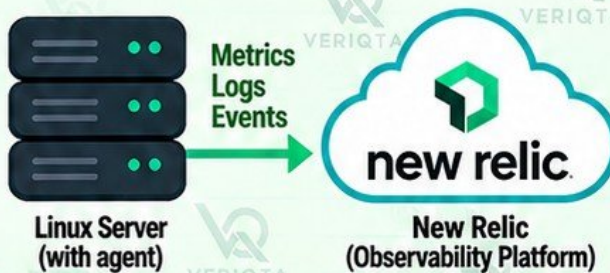


@veriqta

Installing the New Relic Infrastructure Agent on Linux

What Is the Infrastructure Agent?

The New Relic **Infrastructure Agent** is a lightweight agent that you install on your servers. It collects **system and infrastructure data** such as CPU, memory, disk, network, processes and more, and sends it to New Relic.



Why Use the Infrastructure Agent?

- ✓ Monitor server health in real time.
- ✓ Track CPU, memory, disk, network and processes.
- ✓ See system metrics alongside application data.
- ✓ Troubleshoot performance issues faster.
- ✓ Get full visibility of your infrastructure.
- ✓ Works for physical servers, VMs and cloud instances.

Prerequisites



- 1 Linux server (RHEL, CentOS, Ubuntu, Amazon Linux, etc.).
- 2 Sudo or root access.
- 3 Outbound internet access to New Relic endpoints.
- 4 A New Relic account and an Ingestion License Key.
- 5 At least 1 GB RAM (recommended).

Step 1: Get Your Ingestion License Key

- 1 Log in to your New Relic account.
- 2 Go to **All capabilities** → **Infrastructure**.
- 3 Click **Add data** → **Install New Relic agent**.
- 4 Copy your **Ingestion License Key** (starts with NRIA-).

Step 2: Install the Agent (Linux)

Download and install using the official script.

```
curl -o- https://download.newrelic.com/install.sh | sudo bash

sudo /usr/local/bin/newrelic-install install \
  --license-key YOUR_LICENSE_KEY
```

! Replace **YOUR_LICENSE_KEY** with your actual key.

Step 3: Check Agent Status

```
sudo systemctl status newrelic-infra
```

You should see: **active (running)**

Step 4: Verify Data Is Reporting

In the New Relic UI:

- 1 Go to **Infrastructure** → **Hosts**.
- 2 Wait a few minutes (usually 2–5 minutes).
- 3 You should see your Linux host listed with metrics.

Useful Commands

```
sudo systemctl status newrelic-infra    - Check agent status
sudo systemctl restart newrelic-infra    - Restart the agent
sudo /usr/local/bin/newrelic-infra status - Agent details
sudo tail -f /var/log/newrelic-infra/newrelic-infra.log - View agent logs
```



Junior Engineer Tip

After installation, always verify in the New Relic UI that your host is reporting data. If it does not appear, check the agent logs and your network access.

Common Issues and Fixes

Issue	How to Fix
Agent not running	Check logs and restart the agent.
Host not appearing	Wait a few minutes, check internet access and license key.
Permission denied	Use sudo or root user.
No data received	Verify outbound firewall rules.



Production Awareness

The infrastructure agent gives you critical visibility into your servers. It helps you detect issues early, reduce downtime and understand the real cause of performance problems.



@veriqta



@veriqta



@veriqta



@veriqta



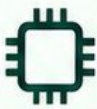
@veriqta



@veriqta

Monitoring Linux CPU, Memory, Disk, and Network

1 CPU Monitoring



CPU (Central Processing Unit) runs your applications. High CPU usage can make applications slow or unresponsive.

Useful Commands

top
Real-time CPU usage

htop
Interactive view (if installed)

mpstat -P ALL 1
Per CPU usage

nproc
Number of CPU cores

Sample Output (top)

```
%Cpu(s): 15.3 us, 5.1 sy, 0.0 ni,
78.9 id, 0.7 wa, 0.0 hi, 0.0 si
```

PID	USER	%CPU	%MEM	COMMAND
1234	app	85.2	2.1	java
5678	root	12.5	0.8	nginx
9012	ubuntu	5.1	1.0	python

How to Interpret

- ✓ %us = CPU used by apps
- ✓ %sy = CPU used by system
- ✓ %id = idle CPU (free)
- ✓ High %cpu means the server is under load.
- ✓ Find the process using high CPU and investigate.

2 Memory Monitoring



Memory (RAM) is used by applications, the OS, and services. When memory is full, applications become slow or may crash.

Useful Commands

free -h
Memory usage (human readable)

vmstat 1
Memory and system stats

top or htop
Real-time memory usage

cat /proc/meminfo
Detailed memory information

Sample Output (free -h)

	total	used	free	buff/cache
Mem:	7.7G	5.1G	1.2G	1.4G
Swap:	2.0G	0B	2.0G	

How to Interpret

- ✓ Used = memory currently in use.
- ✓ Free = available memory.
- ✓ Buff/cache = memory used for caching (can be reused).
- ✓ If used memory is always high, check which processes are using it.

3 Disk Monitoring



Disk stores your operating system, application files, logs, and data. High disk usage can cause application failures and slow performance.

Useful Commands

df -h
Disk usage per filesystem

du -sh /path
Size of a directory

lsblk
List block devices

iostat -x 1
Disk I/O and performance

Sample Output (df -h)

Filesystem	Size	Used	Avail	Use%
/dev/sda1	50G	20G	28G	42%
/dev/sda2	200G	180G	10G	95%
/dev/sdb1	100G	40G	55G	43%

How to Interpret

- ✓ Use% shows how full the disk is.
- ✓ Above 80% is considered high.
- ✓ If disk is full, applications may fail to write logs or data.
- ✓ Check large files and clean unnecessary data.

4 Network Monitoring



Network allows your server to communicate with other services and the internet. High network usage or errors can affect application performance.

Useful Commands

ip -s link
Network interface statistics

ss -tunap
Active network connections

netstat -i
Interface statistics (if available)

iftop or nethogs
Real-time network usage (if installed)

Sample Output (ss -tunap)

State	Local Address	PID/Program
LISTEN	0.0.0.0:80	nginx
LISTEN	0.0.0.0:22	sshd
ESTAB	10.0.0.5:5432	java
ESTAB	10.0.0.5:80	nginx

How to Interpret

- ✓ See which ports are open and which processes are connected.
- ✓ Look for many unexpected connections.
- ✓ Check for high network traffic or errors on interfaces.
- ✓ Investigate if your application cannot reach external services.

How Resource Metrics Affect Applications

- High CPU** → Slow response time, timeouts, application errors
- High Memory** → Applications become slow or crash (Out of Memory)
- High Disk Usage** → Unable to write logs, store data or start services
- Network Issues** → Failed API calls, slow database connections, service downtime



Junior Engineer Tip

Always check all four resources together. A slow application can be caused by CPU, memory, disk I/O or network issues.



Real World Example

If your web application is slow, check CPU for high usage, memory for low available RAM, disk for 100% usage and network for connection issues.

Troubleshooting Workflow

- 1 Identify the problem (slow app, errors, downtime).
- 2 Check CPU, memory, disk and network metrics.
- 3 Find the resource under pressure.
- 4 Identify which process or service is causing it.
- 5 Take action (optimize, restart, increase resources).
- 6 Verify that the issue is resolved.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Monitoring Linux Processes and Infrastructure Health

1 What Is Process Monitoring?



Process monitoring tracks the programs running on your Linux server. It helps you identify **high resource usage**, **failing processes**, and **unusual behavior** before they affect application performance.

2 Why It Matters?

- ✓ Find runaway or stuck processes.
- ✓ Detect memory or CPU leaks.
- ✓ Keep critical services running.
- ✓ Investigate slow or unresponsive servers.
- ✓ Understand the impact on application performance.

3 Common Tools and Commands

Command	What It Does
<code>ps aux</code>	Shows all running processes.
<code>top / htop</code>	Real-time process view.
<code>pidof <name></code>	Find process ID.
<code>pgrep <name></code>	Find process by name.
<code>kill <PID></code>	Stop a process.
<code>systemctl status</code>	Check service status.
<code>journalctl -u</code>	View service logs.

4 Viewing Running Processes

```
[user@server ~]$ ps aux --sort=-%cpu | head
```

USER	PID	%CPU	%MEM	VSZ	RSS	COMMAND
root	2431	85.2	1.5	412028	62300	java
ubuntu	5672	25.4	2.1	287432	81244	python
ubuntu	4310	12.1	1.0	162312	40120	nginx
root	781	5.3	0.5	102312	20488	mysqld
ubuntu	9912	3.1	0.3	18240	8120	node



Look for unexpectedly high CPU or memory usage. Identify the process and investigate.

5 Real-Time Monitoring with top/htop

```
[user@server ~]$ top
```

top - 15:30:12 up 3 days, 2:12, 2 users
Tasks: 210 total, 2 running, 208 sleeping
%Cpu(s): 72.3 us, 5.1 sy, 0.0 ni, 22.1 id
MiB Mem : 7979.7 total, 1200.3 free,
MiB Swap: 2048.0 total, 2048.0 free

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM
2431	root	20	0	412028	62300	85.2	1.5
5672	ubuntu	20	0	287432	81244	25.4	2.1
4310	ubuntu	20	0	162312	40120	12.1	1.0
781	root	20	0	102312	20488	5.3	0.5



Use **top** or **htop** for real-time process activity. Press **q** to exit.

6 Managing Processes

Find a process by name:

```
pgrep -fl nginx
```

Find the process ID:

```
pidof nginx
```

Stop a process (gracefully):

```
kill <PID>
```

Force stop a process:

```
kill -9 <PID>
```

Restart a service:

```
sudo systemctl restart nginx
```

7 Checking Service Health

```
[user@server ~]$ systemctl status nginx
```

● nginx.service - A high performance web server
Loaded: loaded (/usr/lib/systemd/system/nginx.service)
Active: **active (running)** since Mon 2026-10-12 10:15:21
Main PID: 4310 (nginx)
Tasks: 5 (limit: 4915)
Memory: 40.1M
CPU: 1.234s
CGroup: /system.slice/nginx.service
├─4310 nginx: master process
├─4311 nginx: worker process
└─4312 nginx: worker process

8 Viewing Service Logs

```
[user@server ~]$ journalctl -u nginx -n 10
```

Oct 12 15:20:11 server nginx[4310]: starting nginx...
Oct 12 15:20:11 server nginx[4310]: configuration test is successful
Oct 12 15:20:11 server systemd[1]: Started nginx service.
Oct 12 15:25:33 server nginx[4311]: client 192.168.1.50 - "GET /" 200



Check logs for errors, crashes or repeated restarts.

9 New Relic Process & Host View

new relic

Infrastructure

Processes

Containers

Network

Storage

More

Processes

Search processes

Process name	CPU %
java	85.2
nginx	12.1
python	6.3
mysqld	4.8
node	3.1

10 Host Availability and Health

Host overview

● Host is reporting data
Linux | 2 vCPU | 8 GB RAM

- ✓ Host up/down status
- ✓ CPU, memory, disk, network usage
- ✓ Running processes
- ✓ Open ports
- ✓ System load and uptime



Junior Engineer Tip

Always check processes, resources and service health together. A slow or unresponsive server is usually caused by a combination of high resource usage, failed services or application issues.

11 Investigating a Slow or Unresponsive Server

- 1 Check host availability in New Relic.
- 2 Look at CPU, memory, disk and network metrics. Identify high resource processes.
- 3 Check service health and logs.
- 4 Find recent changes (deployments, config updates).
- 5 Take action (restart, increase resources, fix the issue).



Real World Example

A web server becomes slow. You find a Java process using high CPU, nginx is still running, and disk usage is 95%. After clearing logs and restarting the Java service, the server returns to normal.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Introduction to Application Performance Monitoring (APM)

1 What Is APM?



Application Performance Monitoring (APM) helps you monitor and understand the performance and behaviour of your applications in real time, so you can detect issues early and keep users happy.

2 What Does APM Monitor?



Response time
How long requests take.



Throughput
Number of requests handled.



Error rate
Failed requests (4xx, 5xx).



Transaction behaviour
Detailed breakdown of what happens inside a request.

3 Why APM Is Important

- ✓ Helps you find performance issues quickly.
- ✓ Shows you the real user impact.
- ✓ Identify slow endpoints and failing services.
- ✓ Understand dependencies between services.
- ✓ Improve application reliability and user experience.
- ✓ Reduce downtime and faster troubleshooting.

4 Key APM Metrics

Metric	What It Shows	Example	Why It Matters
Response Time	Time taken to complete a request.	250 ms	High response time can indicate slow code, database or external services.
Throughput	Number of requests handled per minute.	1,200 req/min	Shows application load and capacity.
Error Rate	Percentage of failed requests.	2%	High error rate affects users and business.
Transaction Duration	Time spent in each part of a transaction.	DB: 150 ms External API: 300 ms	Helps you find the slow component.

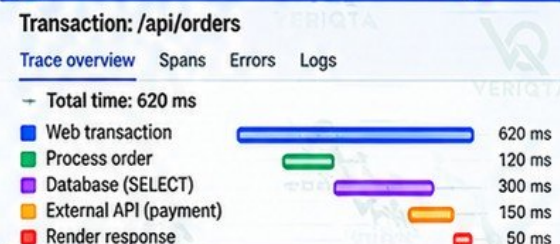
5 Example APM Dashboard (New Relic)



6 How APM Works (High Level)



7 Transaction Breakdown Example



8 Common APM Use Cases

- ✓ Find slow endpoints and optimize them.
- ✓ Investigate high error rates.
- ✓ Understand the impact of a database or external service.
- ✓ Identify performance degradation before users complain.
- ✓ Monitor microservices and distributed transactions.
- ✓ Improve application performance and reliability.

9 APM vs Infrastructure Monitoring

APM (Application Performance Monitoring)	Infrastructure Monitoring
✓ Focuses on application performance. ✓ Monitors transactions, response time, error rate and traces. ✓ Shows code-level and service-level details. ✓ Helps you find slow or failing parts of your application.	✓ Focuses on servers, containers and infrastructure. ✓ Monitors CPU, memory, disk, network and host health. ✓ Does not show application transactions. ✓ Used together with APM for complete visibility.



Junior Engineer Tip

Always look at APM data together with infrastructure data. A slow application can be caused by application code, database, external services or a resource issue on the server.



Real World Example

A web application is slow. APM shows high response time for /api/orders and traces reveal a slow database query. After optimizing the query, response time returns to normal.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Instrumenting an Application with a New Relic APM Agent

1 What Is Application Instrumentation?



Instrumentation means adding a New Relic APM agent to your application so it can collect and send telemetry (transactions, traces, errors, metrics) to New Relic.

2 How the APM Agent Works

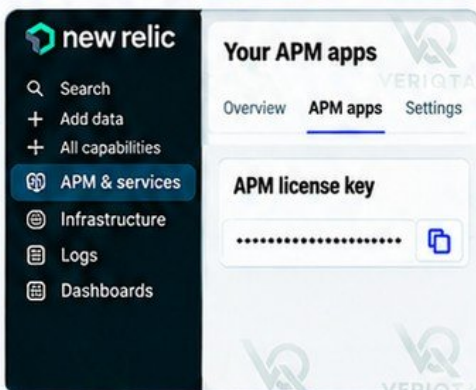


3 Prerequisites

- ✓ A New Relic account.
- ✓ APM license key (ingestion key).
- ✓ Supported application runtime (Java, Node.js, Python, .NET, Ruby, PHP, etc.).
- ✓ Network access to New Relic endpoints.
- ✓ Permission to modify application configuration.

4 Get Your APM License Key

- 1 Log in to your New Relic account.
- 2 Go to [All capabilities](#) → [APM & services](#).
- 3 Click [Your APM apps](#) → Copy your [License key](#).



5 Install and Configure the Agent

Example: Node.js Application

```

bash
# Install the New Relic Node.js agent
npm install newrelic

# Edit newrelic.js with your license key
license_key: process.env.NEW_RELIC_LICENSE_KEY,
app_name: ['my-node-app']

# Start your application
node app.js
  
```

Example: Java Application

```

bash
# Add New Relic agent (download agent JAR)
# Run with the Java agent
java -javaagent:newrelic.jar -jar app.jar
  
```

Example: Python Application

```

bash
# Install the agent
pip install newrelic

# Configure using environment variable
export NEW_RELIC_LICENSE_KEY=YOUR_LICENSE_KEY

# Run your app (e.g. Flask)
newrelic-admin run-program python app.py
  
```

6 Secure License Key Handling

- ✓ Use environment variables (never hardcode in code).
- ✓ Restrict access to the key.
- ✓ Use different keys for different environments (dev, staging, production).
- ✓ Rotate keys if exposed.
- ✓ Store in a secrets manager (AWS Secrets Manager, Azure Key Vault, etc.).

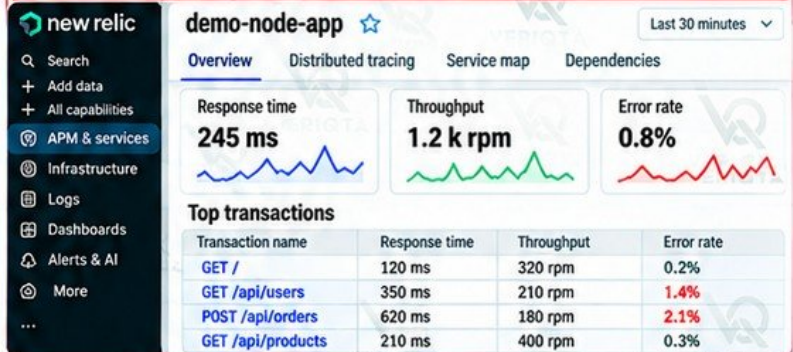
! Do Not:

- ✗ Commit the license key to Git.
- ✗ Share the key publicly.
- ✗ Use the same key for multiple unrelated applications.

7 Verify That Data Is Reporting

- 1 Restart your application.
- 2 Wait a few minutes for data to appear.
- 3 In New Relic, go to [APM & services](#) and select your application.
- 4 Check the following:
 - Application overview shows data.
 - Transactions are listed.
 - Response time and throughput graphs are visible.
 - Errors (if any) are captured.
 - Service map shows dependencies (if applicable).

8 APM Application Overview (Example)



9 Common Issues and Fixes

Issue	How to Fix
No data in New Relic	Check license key, restart application, verify network access.
App name not showing	Check agent configuration and app_name.
Missing transactions	Ensure agent is loaded before the application starts.
High memory/CPU	Use a supported agent version and check configuration.
Connection errors	Verify firewall/proxy settings and New Relic endpoints.

10 Junior Engineer Tip



After instrumenting, generate some traffic to your application (e.g. use a browser, curl or a load tool) so you can quickly see transactions, response times and service map data in New Relic.



Real World Example

You instrument a Node.js web application. After deployment, you see transactions, response time, error rate and database dependencies in New Relic within a few minutes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Understanding Transactions, Latency, Throughput, and Error Rates

1 What Are Transactions?



A transaction is a single request that a user makes to your application (e.g. a web request, API call or background job).

2 Key Metrics Explained

Metric	What It Means	Example	Why It Matters
Latency	Time taken to complete a request.	250 ms	High latency means slow responses.
Throughput	Number of requests handled per time.	1,200 req/min	Shows traffic and application capacity.
Error Rate	Percentage of failed requests.	2%	Indicates issues and affects user experience.
Transactions	Individual requests to your application.	/api/login	Helps you monitor specific endpoints.

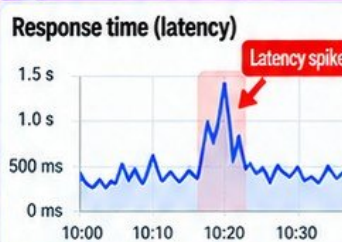
3 Why These Metrics Matter

- ✓ Identify slow requests and endpoints.
- ✓ Detect traffic spikes early.
- ✓ Find failing services and errors.
- ✓ Understand the impact on users.
- ✓ Help with troubleshooting and root cause analysis.
- ✓ Improve application performance and reliability.

4 Viewing Transactions in New Relic



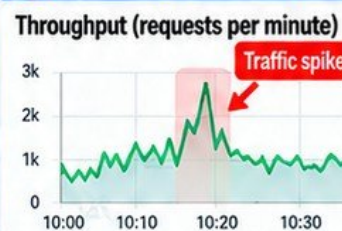
5 Latency Analysis



How to Investigate High Latency

- 1 Check which endpoint is slow.
- 2 Look at traces to find where time is spent.
- 3 Identify database or external service delays.
- 4 Compare with deployments or changes.

6 Throughput Analysis



How to Investigate Traffic Spikes

- 1 Check which endpoints received more traffic.
- 2 See if it is expected (e.g. marketing, event).
- 3 Check error rate and latency during the spike.
- 4 Verify if the application scaled correctly.

7 Error Rate Analysis



8 Transaction Traces (Waterfall View)

Trace details	Span table	Logs	Errors
GET /api/orders	620 ms	200 spans	
WebTransaction	620 ms		
Expressjs/route	120 ms		
Database/select	320 ms		
External API	150 ms		
Response	30 ms		

9 Common Causes and Fixes

Issue	Possible Cause	How to Fix
High latency	Slow database query	Optimize query, add index, cache.
Low throughput	Server under-provisioned	Scale resources, check bottlenecks.
High error rate	Application bug	Check logs, fix code, review deploy.
Slow endpoint	External service delay	Investigate external dependency.
Traffic spike	Increased user activity	Scale application, enable autoscaling.

10 Junior Engineer Tip



Always look at latency, throughput and error rate together. A high error rate or high latency during a traffic spike usually indicates a real issue that affects users.



Real World Example

During a product launch, traffic increased 5x. Latency for /api/orders went from 200 ms to 1.2 s and error rate increased to 5%. The team scaled the application and optimized the database query, restoring normal performance.

Distributed Tracing: Following a Request Across Services

1 What Is Distributed Tracing?



Distributed tracing follows a single request as it moves through multiple services. It shows you each step (spans), the time spent in each service, and where delays or errors occur.

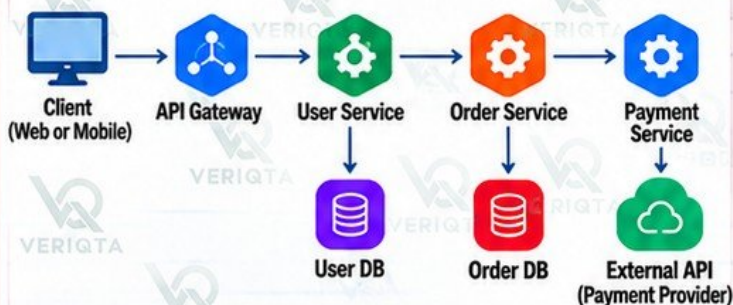
2 Key Concepts

Trace	Complete journey of a single request across services.
Span	A single operation within a service (e.g. DB query, API call).
Service	Application or component that handles a part of the request.
Parent Span	The operation that started another span.
Child Span	An operation created by a parent span.
Trace ID	Unique ID for the whole request.
Span ID	Unique ID for a single span.

3 Why Tracing Matters

- ✓ Find the exact service causing a delay.
- ✓ Understand dependencies between services.
- ✓ Identify slow database queries and external API calls.
- ✓ Troubleshoot errors across multiple services.
- ✓ Improve performance, reliability and user experience.

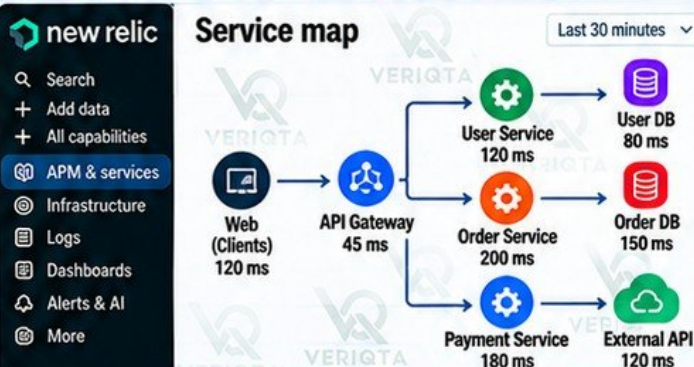
4 Example Application Flow (Microservices)



5 Sample Trace in New Relic (Waterfall View)



6 Trace with Service Map (New Relic)



7 Span Details Example

Overview	Attributes	Logs	Errors
Service	order-service		
Operation	SELECT orders		
Duration	150 ms		
Timestamp	Oct 12, 2026 10:24:31		
Status	Success		
Parent span	a1b2c3d4e5f6		
Span ID	b7c8d9e0f1a2		

Common Span Attributes	
Attribute	Example
http.method	GET
http.url	/api/orders
db.statement	SELECT ...
peer.service	payment-service
host.name	ip-10-0-1-23
error	false
status.code	200

8 How to Investigate a Slow Request

- Find a slow transaction in New Relic.
- Open the trace and look at the waterfall view.
- Identify which service or span has the highest duration.
- Check for database queries, external API calls or dependent services.
- Look at logs and errors for the slow span.
- Compare with normal traces to find the change.
- Take action (optimize code, fix query, increase resources or fix the dependency).



Junior Engineer Tip

Always follow the trace from the slowest span. The span with the highest duration usually shows you the real cause of the issue.

9 Common Causes of High Latency

Cause	Description
Slow database query	Inefficient query or missing index.
External API delay	Third-party service is slow.
High load on a service	Not enough resources or scaling.
Network latency	Network or DNS issues.
Large payload	Request or response too large.
Synchronous calls	Waiting for dependent services.
Code performance	Inefficient code or blocking operations.



Real World Example

A /api/orders request takes 1.8 seconds. Tracing shows the payment-service call to the external API takes 1.2 seconds. After caching the response, the total request time drops to 250 ms.

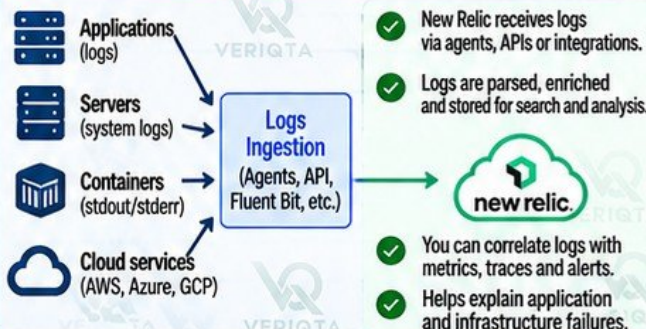
Centralized Log Monitoring with New Relic

1 What Is Log Monitoring?



Log monitoring collects, stores and analyzes logs from applications, servers, containers and other services in one place so you can quickly find the cause of issues and understand system behavior.

2 How Logs Reach New Relic



3 Common Log Sources

- Application logs (e.g. Java, Node.js, Python)
- System logs (e.g. /var/log/syslog, /var/log/messages)
- Web server logs (e.g. Nginx, Apache)
- Container logs (e.g. Docker, Kubernetes)
- Cloud service logs (e.g. AWS, Azure, GCP)
- Database logs (e.g. MySQL, PostgreSQL)

4 Viewing and Searching Logs in New Relic

new relic

- Search
- Add data
- All capabilities
- APM & services
- Infrastructure
- Logs**
- Dashboards
- Alerts & AI
- More

Logs Explorer Last 30 minutes

level:error Run

Add filter service hostname level ...

Timestamp	Level	Service	Message
Oct 12 10:23:11	ERROR	order-service	Database connection failed
Oct 12 10:23:10	WARN	api-gateway	High response time: 1200ms
Oct 12 10:22:59	INFO	user-service	User login successful
Oct 12 10:22:41	ERROR	payment-service	Payment timeout after 30s
Oct 12 10:22:10	INFO	nginx	GET /api/orders 200

5 Useful Log Search Queries

Goal	Query Example
Find errors	level:error
Find logs for a service	service:order-service
Filter by hostname	hostname:ip-10-0-1-23
Search by message text	message:"connection failed"
Find logs in a time range	level:error SINCE 1 hour ago
Filter by log level	level:warn OR level:error
Find logs for an endpoint	message:"/api/orders"
Filter by container	container.name:frontend
Find database errors	message:"SQL" AND level:error
Exclude noisy logs	NOT message:"health check"

6 Example: Investigating an Application Failure

- User reports they cannot place an order.
- Check APM for high error rate.
- Open Logs Explorer and search for errors.
- Filter by service, endpoint or trace ID.
- Look at log messages and timestamps.
- Identify the root cause (e.g. database error).
- Take action (restart service, fix code, scale resources).

service:order-service level:error Last 15 minutes

```

Oct 12 10:21:33 ERROR order-service
Database connection failed:
java.sql.SQLException: Connection refused
Oct 12 10:21:33 ERROR order-service
Failed to execute query: SELECT * FROM orders
Oct 12 10:21:34 INFO order-service
Retrying database connection (attempt 1/3)
Oct 12 10:21:36 INFO order-service
Database connection established
  
```

7 Correlating Logs with Traces and Metrics

Overview **Logs** Traces Metrics Infrastructure

Trace ID: 9f3e2c1a8b74e5f View trace

Service	Duration	Error rate
order-service	1.8 s	12%

Related logs for this trace

Timestamp	Level	Service	Message
10:20:11	INFO	order-service	Received request /api/orders
10:20:12	ERROR	order-service	Database timeout
10:20:13	ERROR	order-service	Failed to execute query
10:20:14	INFO	order-service	Request failed with 500

8 Common Causes Found in Logs

Cause	Example Log Message
Database issues	java.sql.SQLException: Connection refused
External API failure	Error calling external service: timeout
High traffic / overload	Rejected execution (queue full)
Configuration error	Invalid configuration value: port
Out of memory	java.lang.OutOfMemoryError: Java heap space
Permission issue	Access denied: permission denied
Application exception	NullPointerException at com.example.Service
Network issue	Connection timed out

9 Best Practices

- Use structured logs (JSON).
- Include key fields (timestamp, level, service, trace ID).
- Set log levels correctly (INFO, WARN, ERROR).
- Keep sensitive data out of logs.
- Use filters and saved views.
- Correlate logs with traces and metrics.
- Set up alerts for critical error patterns.
- Retain logs for the right period based on your needs.

10 Junior Engineer Tip

When troubleshooting an issue, always check logs along with APM traces and infrastructure metrics. Logs often show the exact error message and help you quickly understand what went wrong.

Real World Example
A sudden spike in 500 errors is seen in APM. Logs reveal a database connection error caused by a misconfigured database endpoint. Fixing the configuration and restarting the service restores normal operation.

Introduction to New Relic Query Language (NRQL)

1 What Is NRQL?



New Relic Query Language (NRQL) is a SQL-like language used to query and analyze telemetry data (metrics, logs, events, traces and more) in New Relic.

2 Key NRQL Clauses

Clause	Purpose
SELECT	Choose the data and function (e.g. count, average, max).
FROM	Specify the data source (event type).
WHERE	Filter the data.
SINCE	Set the time range (e.g. 1 hour ago).
TIMESERIES	Group results into time buckets.
FACET	Group results by a field (e.g. service, host, endpoint).

3 Common Data Sources (Event Types)

	Transaction	- Application transactions (APM)
	Span	- Distributed tracing spans
	Log	- Application and system logs
	Metric	- Infrastructure and custom metrics
	SystemSample	- Host metrics (CPU, memory, disk, etc.)
	ProcessSample	- Process-level metrics
	NetworkSample	- Network metrics
	ContainerSample	- Container metrics (Docker/Kubernetes)

4 Basic NRQL Query Structure

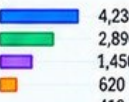
```
SELECT <function>(<attribute>)
FROM <event_type>
WHERE <condition>
SINCE <time_range>
TIMESERIES <interval>
FACET <attribute>
```

Example structure

```
SELECT average(duration)
FROM Transaction
WHERE appName = 'demo-app'
SINCE 1 hour ago
TIMESERIES 5 minutes
FACET name
```

5 Run a Query in New Relic

6 Query Examples

Use Case	NRQL Query	Result (Example)										
Total requests (count)	<pre>SELECT count(*) FROM Transaction SINCE 1 hour ago</pre>	Total requests 12,430 										
Average response time	<pre>SELECT average(duration) FROM Transaction SINCE 1 hour ago TIMESERIES 5 minutes</pre>	Average duration (ms) 										
Error rate	<pre>SELECT percentage(count(*), WHERE error IS true) FROM Transaction SINCE 1 hour ago</pre>	Error rate (%) 2.4% 										
Top endpoints by traffic	<pre>SELECT count(*) FROM Transaction SINCE 1 hour ago FACET name LIMIT 5</pre>	Top 5 endpoints  <table><tr><td>/api/orders</td><td>4,230</td></tr><tr><td>/api/products</td><td>2,890</td></tr><tr><td>/api/users</td><td>1,450</td></tr><tr><td>/health</td><td>620</td></tr><tr><td>/login</td><td>410</td></tr></table>	/api/orders	4,230	/api/products	2,890	/api/users	1,450	/health	620	/login	410
/api/orders	4,230											
/api/products	2,890											
/api/users	1,450											
/health	620											
/login	410											

7 Filter and Analyze Data

Example	Description
<pre>SELECT count(*) FROM Transaction WHERE appName = 'demo-app' SINCE 1 hour ago</pre>	Filter by application name.
<pre>SELECT average(duration) FROM Transaction WHERE name = '/api/orders' SINCE 1 hour ago</pre>	Filter by specific endpoint.
<pre>SELECT count(*) FROM Transaction WHERE error IS true SINCE 1 hour ago FACET name</pre>	Find errors grouped by endpoint.
<pre>SELECT average(duration) FROM Transaction WHERE appName = 'demo-app' SINCE 1 hour ago TIMESERIES 1 minute</pre>	See response time over time.

8 Use FACET and TIMESERIES

Group by service (FACET)

```
SELECT count(*)
FROM Transaction
SINCE 1 hour ago
FACET appName
```

web-app	4,230
order-service	2,890
payment-service	1,450
user-service	620

Time series analysis (TIMESERIES)



9 Junior Engineer Tip



Start with simple queries using SELECT, FROM and SINCE, then add WHERE, TIMESERIES and FACET as needed to investigate specific issues.



Real World Example

An API endpoint became slow. Use NRQL to compare response time before and after a deployment, facet by endpoint, and check the error rate to see the full impact.

Building Your First New Relic Dashboard

1 What Is a New Relic Dashboard?



A New Relic dashboard lets you visualize important metrics from your applications and infrastructure in one place so you can quickly monitor health, spot issues and track performance over time.

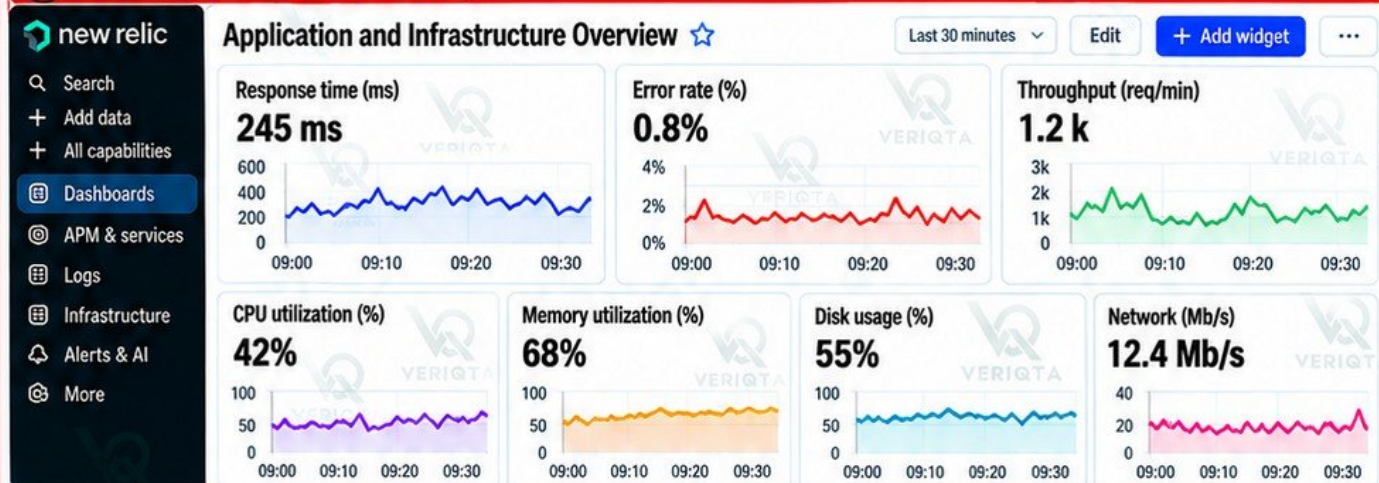
2 Dashboard Goals

- ✓ Show application response time (latency).
- ✓ Show error rate.
- ✓ Show throughput (requests per minute).
- ✓ Show Linux resource utilization (CPU, memory, disk and network).
- ✓ Use NRQL queries.
- ✓ Make it easy to understand.
- ✓ Help you detect issues quickly.

3 Create a New Dashboard

- 1 Log in to your New Relic account.
- 2 Go to **Dashboards** → **+ New dashboard**.
- 3 Choose a name (e.g. Application and Infrastructure Overview).
- 4 Add a description (optional).
- 5 Click **Create**.
- 6 Start adding widgets (charts, tables, visualizations).

4 Example Dashboard (Overview)



5 NRQL Queries for Dashboard Widgets

Widget	NRQL Query	Visualization
Application response time	<code>SELECT average(duration) FROM Transaction SINCE 30 minutes ago</code>	Line chart
Error rate	<code>SELECT percentage(count(*), WHERE error IS true) FROM Transaction SINCE 30 minutes ago</code>	Line chart
Throughput (requests per minute)	<code>SELECT count(*) FROM Transaction SINCE 30 minutes ago TIMESERIES 1 minute</code>	Line chart
CPU utilization (Linux)	<code>SELECT average(cpuPercent) FROM SystemSample SINCE 30 minutes ago</code>	Line chart
Memory utilization (Linux)	<code>SELECT average(memoryPercent) FROM SystemSample SINCE 30 minutes ago</code>	Line chart
Disk usage (Linux)	<code>SELECT average(diskUsedPercent) FROM SystemSample SINCE 30 minutes ago</code>	Line chart
Network traffic (Linux)	<code>SELECT average(networkReceiveBytesPerSecond) FROM SystemSample SINCE 30 minutes ago</code>	Line chart

6 Widget Types You Can Use

-  **Line chart** – Show trends over time (latency, throughput).
-  **Bar chart** – Compare values (e.g. top services).
-  **Area chart** – Show volume over time.
-  **Table** – Show top endpoints, hosts, errors.
-  **Billboard** – Show a single key value (e.g. current error rate).
-  **Pie chart** – Show distribution (e.g. errors by service).
-  **Heatmap** – Show patterns over time.
-  **Sankey** – Show service relationships (APM).

7 Best Practices

- ✓ Use meaningful titles and clear units.
- ✓ Group related metrics (APM, infrastructure, errors).
- ✓ Set the right time range.
- ✓ Keep the dashboard simple and focused.
- ✓ Add filters (e.g. environment, service name).
- ✓ Share with your team (view or edit access).

8 Junior Engineer Tip

Start with a simple dashboard and add more widgets over time. Focus on the most important metrics: latency, error rate, throughput and infrastructure health.

9 Real World Example

When response time increases and error rate spikes, you can quickly check the dashboard to see if the application or the Linux host is under resource pressure (high CPU, memory or disk usage).

Creating Alerts and Understanding Alert Conditions

1 What Are Alerts in New Relic?



Alerts notify you when a metric, log or event crosses a defined threshold, helping you quickly detect and respond to potentially unhealthy systems before users are impacted.

2 Key Components

Component	Description
Alert policy	Group of related alert conditions for a service or team.
Condition	Defines what to monitor, the threshold and when to alert.
Threshold	The value that triggers the alert (e.g. CPU > 80%).
Evaluation window	Time period used to evaluate the condition (e.g. 5 minutes).
Signal	Metric, log, event or query (NRQL).
Notification channel	Where alerts are sent (email, Slack, PagerDuty, etc.).

3 Types of Alert Conditions

	Metric alert	Threshold on New Relic metrics (e.g. CPU, memory, response time).
	NRQL alert	Custom condition using NRQL queries.
	Log alert	Triggers when specific log patterns or error messages occur.
	Event alert	Based on New Relic events (e.g. deployment, host down).
	Synthetic alert	Based on browser or API monitor checks.
	Anomaly alert	Uses machine learning to detect unusual behavior.

4 Create an Alert Policy

new relic

- Search
- + Add data
- + All capabilities
- Alerts & AI**
- Incidents
- Alert conditions
- Alert policies**
- Notification channels
- Workflows
- Destinations
- AI monitoring
- More

Alert policies + New alert policy

Policy name
Web Application Alerts

Policy description (optional)
Alerts for application and infrastructure health

Incident preference
☒ Create an incident for each condition
☐ Create a single incident for multiple conditions

Notification channels
 Email (team@veriqta.com) Slack (veriqta-alerts) ×
 + Add channel

5 Create an Alert Condition (Metric Example)

Create alert condition

1 Define condition 2 Set thresholds 3 Add details

Select a signal
Metric

Choose a metric
SystemSample: cpuPercent

Aggregation
Average

Condition name
High CPU usage

Run this condition for
5 minutes

Set thresholds

- Critical threshold**
When the value is above 80 %
- Warning threshold**
When the value is above 60 %

Preview

6 Common Alert Examples

Use Case	Signal / Query	Condition	Purpose
High CPU usage	SystemSample: cpuPercent	> 80% for 5 minutes	Detect high server resource usage.
High memory usage	SystemSample: memoryPercent	> 80% for 5 minutes	Detect memory pressure.
High response time	Transaction duration (APM)	> 1,000 ms for 5 minutes	Detect slow application performance.
High error rate	Transaction error percentage	> 5% for 5 minutes	Detect application errors early.
Low throughput	NRQL (requests)	< 100 req/min for 5 minutes	Detect traffic drops.
Log error pattern	Logs (message)	Contains "ERROR" > 10 times in 5 min	Detect repeated errors.

7 NRQL Alert Condition Example

NRQL query (High error rate) Copy

```
SELECT percentage(count(*), WHERE error IS true)
FROM Transaction
WHERE appName = 'demo-app'
SINCE 5 minutes ago
```

Setting	Value
Signal type	NRQL query
Threshold (critical)	> 5%
Threshold (warning)	> 2%
Evaluation window	5 minutes
Aggregation	Average
Purpose	Detect high application error rate.

8 Alert Lifecycle



9 Best Practices

- Set meaningful thresholds.
- Use appropriate evaluation windows.
- Avoid alert fatigue (don't set too many alerts).
- Use different severities (warning and critical).
- Include useful context in alert messages.
- Group related conditions in a policy.
- Test your alerts to ensure they work.

10 Junior Engineer Tip

Start with simple metric alerts (e.g. CPU or error rate) and increase complexity as you become familiar with NRQL and your application.

11 Real World Example

A sudden increase in error rate triggered an alert. The team investigated and found a failed database connection, restored the database and the alert cleared after 5 minutes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Understanding Incidents, Issues, and Notifications

1 Key Concepts



Alert Event

A single alert condition is violated (e.g. CPU > 80%).



Incident

A group of related alert events that occur at the same time.



Issue

A longer-lived problem created from an incident for tracking and resolution.



Notification

Sends alerts to the right people (email, Slack, PagerDuty, webhook, etc.).

2 How It Works



Alert Events

Multiple alerts triggered (e.g. high CPU, high memory)



Incident

Related alert events are grouped into one incident based on correlation and time.



Issue

An incident can create an issue for tracking, ownership and resolution.



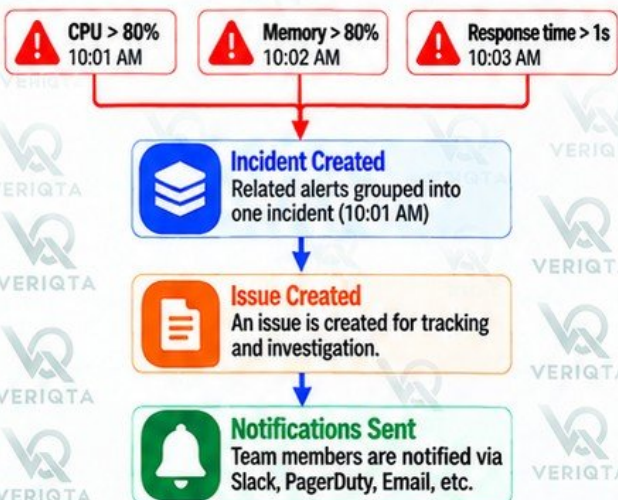
Notifications

Alerts, incidents or issues are routed to the right responders via configured channels.

3 Incident vs Issue vs Alert Event

Item	What It Is	When It Happens	Example
Alert Event	A single violation of an alert condition.	When a metric, log or event breaches a threshold.	CPU > 80% for 5 minutes.
Incident	A group of related alert events.	When multiple alerts are correlated in a time window.	CPU high + Memory high + Disk high at the same time.
Issue	A longer-lived problem for tracking and resolution.	Created manually or automatically from an incident.	Create an issue for "Payment Service Outage".
Notification	Sends alerts to responders.	When an alert event, incident or issue triggers a notification channel.	Send to Slack, PagerDuty or email.

4 Example: Related Alert Events → Incident → Issue



5 Example: Incident and Issue in New Relic

- Search
- Add data
- All capabilities
- Dashboards
- APM & services
- Logs
- Infrastructure
- Alerts & AI
- Incidents**
- Issues
- Alert conditions
- Notification channels
- Destinations
- Workflows
- More

Incidents + Create issue

Open (2) Acknowledged (1) Closed (12)

Title	State	Priority	Start time
Payment service degradation	Open	High	Oct 12, 10:01 AM
Database high CPU	Ack	High	Oct 12, 09:44 AM

Incident details

Incident ID: NRI-12345
Start time: Oct 12, 2026 10:01 AM
Related alerts: High CPU, High Memory, High Response time
Linked issue: #ISSUE-5678
[View issue](#)

Timeline

10:01 AM Alert event: CPU > 80%
10:02 AM Alert event: Memory > 80%
10:03 AM Alert event: Response time > 1s
10:03 AM Incident created: NRI-12345
10:05 AM Issue created: #ISSUE-5678
10:05 AM Notifications sent (Slack, PagerDuty)

6 Notification Channels

Channel	What It Does	When to Use
Email	Sends alert notifications to email addresses.	Good for general alerts and backup notifications.
Slack	Sends alerts to a Slack channel or user.	Great for real-time team notifications.
PagerDuty	Creates an incident in PagerDuty and notifies on-call engineers.	Use for critical production incidents (24/7).
Webhook	Sends a JSON payload to a custom endpoint.	Use for integrations with custom tools (e.g. internal systems).
Microsoft Teams	Sends alerts to a Teams channel.	Use if your team uses Microsoft Teams.

7 Best Practices

- Use meaningful alert names and descriptions.
- Group related alerts to reduce noise.
- Set the right thresholds and evaluation windows.
- Route notifications to the right teams (on-call, SRE, DevOps).
- Use incident correlation to avoid alert fatigue.
- Create issues for tracking and post-incident analysis.
- Regularly review and fine-tune alert conditions.

8 Junior Engineer Tip



Don't create too many alerts. Focus on key indicators (CPU, memory, error rate, response time) and make sure alerts are actionable and routed to the right people.

9 Real World Example



A sudden increase in response time triggers multiple alerts (CPU, memory, latency). New Relic groups them into an incident, creates an issue, and notifies the on-call team via Slack and PagerDuty. The team investigates and resolves the issue, and the incident is closed.

Troubleshooting a Slow Production Application

1 Identify the Problem



Users report the application is slow.



Check alerts, incidents and recent changes deployments.



Identify when the issue started and which services are affected.



Understand the impact (e.g. high latency, timeouts, error rates).

2 Check APM Metrics (New Relic)

Web transaction time (ms)

2,850 ms



Throughput (req/min)

320



Error rate (%)

4.2%



3 Check Infrastructure Metrics

CPU utilization (%)

85%



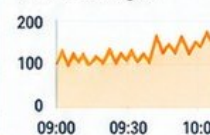
Memory utilization (%)

78%



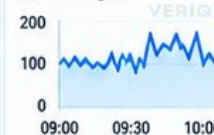
Disk I/O (MB/s)

120 MB/s

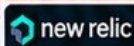


Network (Mb/s)

95 Mb/s



4 Investigate with Distributed Tracing



Trace details

Trace ID: 8f3a2c9e7d4b1a6c

2,850 ms



The order-service is the slowest component (1,950 ms). Click into the span to see more details and related logs.

5 Analyze Application Logs



Logs Explorer

service:order-service level:error

Run

Timestamp	Level	Service	Message
10:02:11	ERROR	order-service	Database query timed out after 30s
10:02:10	WARN	order-service	Slow query detected (2,350 ms)
10:02:08	ERROR	order-service	Connection pool exhausted
10:02:05	INFO	order-service	Retrying database connection (attempt 3)
10:02:01	ERROR	order-service	Query failed: timeout exceeded

6 Investigate Database/External Calls



Check database metrics (CPU, connections, slow queries).



Look for slow queries in logs and traces.



Check external API calls for high latency or failures.



Identify dependency issues (e.g. 3rd party service outage).



Verify if there were recent changes deployments, configuration updates).

7 Identify the Root Cause



Slow database query (e.g. missing index, full table scan).



High resource usage (CPU, memory or disk I/O).



Latency in external service (e.g. payment gateway, third-party API).



Application issue (e.g. inefficient code, thread or connection pool exhaustion).

8 Take Action



Fix the root cause (e.g. optimize query, restart service, scale resources).



Validate the fix by monitoring metrics.



Ensure response time returns to normal.



Check error rate and throughput.



Monitor for recurrence.



Update alerts or thresholds if needed.

9 Example Investigation Timeline



09:50	Alert received: high response time
09:52	Checked APM metrics (latency high)
09:55	Traced slow request (order-service)
10:00	Found slow database query in logs
10:05	Optimized query and restarted service
10:10	Response time back to normal
10:15	Monitored for 30 minutes (stable)

10 Best Practices



Use APM, logs, traces and infrastructure metrics together.



Follow the request path end-to-end.



Compare with baseline/normal behavior.



Check for recent changes.



Document findings and permanent fixes.

11 Junior Engineer Tip



When an application is slow, always start with APM (response time and traces), correlate with logs for errors, and check infrastructure metrics (CPU, memory, disk, network). The combination usually reveals the root cause.

12 Real World Example



An e-commerce application became slow due to a missing database index. Traces showed high latency in the order-service, logs showed slow queries, and database CPU was high. After adding the index, response time dropped from 2.8s to 300ms.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Investigating Application Errors and Service Failures

1 Common Errors and Failures

5xx

HTTP 5xx Errors

Server-side errors (e.g. 500, 502, 503, 504) indicating application or infrastructure issues.



Application Exceptions

Runtime errors, unhandled exceptions, or failed operations in the application.



Service Availability Issues

A service is down, unhealthy or not responding.



Failed Dependencies

Errors from databases, external APIs, message queues, or other services.

2 Analyze Error Rate in New Relic

Error rate (%)

Last 1 hour

5.6%

Top HTTP error codes

Status code	Count	Percentage
500	1,243	56.2%
502	543	24.5%
503	312	14.1%
504	114	5.2%

3 Check Affected Services

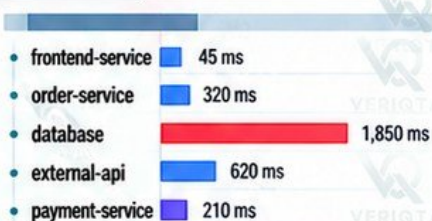
Service	Error rate	Response time	Status
frontend-service	0.4%	220 ms	✓
order-service	8.5%	1,240 ms	!
payment-service	0.3%	180 ms	✓
user-service	0.2%	160 ms	✓
database	2.1%	950 ms	!
external-api	6.7%	1,850 ms	!

4 Investigate with Distributed Tracing



Trace details

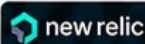
Trace ID: 4f3a9e2c1b7d9e8a



Error

Database query failed
java.sql.SQLException: Connection timeout after 30,000 ms

5 Check Application Logs



Logs Explorer

level:error service:order-service

Run

Timestamp	Level	Service	Message
10:02:14	ERROR	order-service	java.sql.SQLException: Connection timeout after 30,000 ms
10:02:13	ERROR	order-service	Failed to get connection from pool
10:02:12	ERROR	order-service	Error processing order ID=78432
10:02:11	ERROR	order-service	Caused by: java.net.SocketTimeoutException
10:02:10	WARN	order-service	Retrying database connection (attempt 3)

6 Check Infrastructure Health

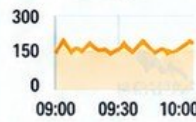
CPU utilization (%)

75%

Memory utilization (%)

82%

Disk I/O (MB/s)

150 MB/s

Network (Mb/s)

110 Mb/s

7 Investigate External Dependencies

Dependency	Status	Response time	Error rate
Database	!	1,850 ms	12.4%
Payment API	!	620 ms	6.1%
Shipping API	✓	220 ms	0.2%
Auth Service	✓	180 ms	0.1%

8 Common Root Causes



Database issues

(slow queries, connection pool exhaustion, high load).



Service overload

(insufficient resources, high traffic).



Application bugs

(unhandled exceptions, memory leaks).



Network or DNS issues

(timeouts, packet loss, service unreachable).



Failed external dependencies

(third-party API or downstream service).

9 Take Action

- ✓ Identify the failing component.
- ✓ Correlate metrics, logs and traces.
- ✓ Check recent changes or deployments.
- ✓ Verify infrastructure and dependencies.
- ✓ Fix the root cause (e.g. optimize query, restart service, scale resources).
- ✓ Monitor to confirm recovery.
- ✓ Update alerts or capacity if needed.

10 Best Practices

- ✓ Always check error rates and response time trends.
- ✓ Use traces to find the exact failing component.
- ✓ Examine logs for detailed error messages.
- ✓ Check infrastructure and external dependencies.
- ✓ Compare with deploy, configuration or traffic changes.
- ✓ Document the root cause and resolution.

11 Junior Engineer Tip



When you see HTTP 5xx errors, start with APM (error rate and traces), then check logs for the exact error message, and finally verify infrastructure and dependencies. Follow the evidence to find the real cause.

12 Real World Example



An e-commerce site returned 503 errors during peak traffic. Traces showed the order-service was slow due to a database connection pool exhaustion. Logs confirmed timeout errors. Increasing the database pool size and optimizing queries resolved the issue, and error rate dropped to 0.1%.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Monitoring Docker, Kubernetes, and Cloud Infrastructure

1 Key Concepts



Docker Monitoring

Track container health, resource usage, logs and application metrics.



Kubernetes Monitoring

Monitor clusters, nodes, pods, deployments, services and events.



Cloud Infrastructure Monitoring

Track compute, storage, network and managed services (e.g. AWS, Azure, GCP).



Why It Matters

Helps you detect issues early, understand resource usage and keep workloads healthy and available.

2 Docker Metrics in New Relic

Container Overview

12 Containers 11 Running 1 Exited 0 Restarting

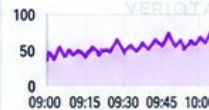
CPU usage (%)

32%



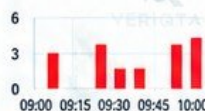
Memory usage (%)

68%



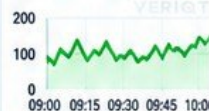
Container restarts

2



Network I/O (MB/s)

120 MB/s



3 Kubernetes Metrics in New Relic

Cluster Overview

3 Nodes 28 Pods 25 Running 3 Not Ready

Node CPU usage (%)

62%



Node memory usage (%)

71%



Pod status



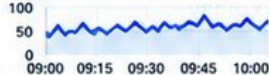
Pod restarts (Top 5)

api-gateway	5
user-service	3
payment-service	2
order-service	2
worker	1

4 Cloud Infrastructure Metrics

EC2 CPU utilization (%)

45%



EC2 memory utilization (%)

70%



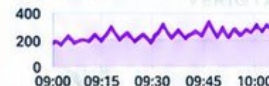
Network in (Mb/s)

180 Mb/s



Network out (Mb/s)

95 Mb/s



Disk read (MB/s)

120 MB/s



Disk write (MB/s)

85 MB/s



5 New Relic Integrations



Docker Integration

Collects container metrics, logs and events.



Kubernetes Integration

Monitors clusters, nodes, pods, deployments, services and events.



AWS Integration

Monitors EC2, ECS, EKS, RDS, load balancers and more.



Azure Integration

Monitors VMs, AKS, databases, network and storage services.



Google Cloud Integration

Monitors GKE, compute engine, cloud storage and other services.

6 Useful Kubernetes Views

new relic

- Search
- Add data
- All capabilities
- Dashboards
- APM & services
- Logs
- Infrastructure
- Kubernetes
- Containers
- Cloud
- Alerts & AI
- More

Kubernetes

Overview Nodes Pods Workloads

Cluster health

Healthy
25 of 28 pods running

Top workloads by CPU

Workload	CPU	Memory
api-gateway	62%	1.2 GB
order-service	48%	980 MB
payment-service	35%	750 MB
user-service	28%	620 MB
worker	18%	400 MB

7 Common Issues and What to Check

Issue	Possible Cause	What to Check
Container restarts	Crash, OOM, misconfiguration	Container logs, restart count, resource usage.
Pod not ready	Image pull error, failed readiness probe	Pod events, logs, probe status.
High CPU usage	Resource limit too low, traffic spike	CPU metrics, application behavior.
High memory usage	Memory leak, large workload	Memory metrics, logs, heap dump.
Service unavailable	Failed dependency, network issue	Traces, logs, downstream service health.
High latency	Database, external API, resource contention	APM traces, logs, infrastructure metrics.
Node not ready	Resource exhaustion, network issue	Node metrics, events, system logs.

8 Best Practices

- Monitor containers, pods and nodes.
- Set resource requests and limits.
- Track restarts and pod status.
- Use dashboards for workload health.
- Set alerts for high CPU, memory, and restart counts.
- Correlate metrics, logs and traces.
- Monitor cloud infrastructure alongside applications.
- Regularly review capacity and optimize resources.
- Document findings and fixes.

9 Junior Engineer Tip



When a pod keeps restarting, check the logs first. The error message usually tells you the real reason (e.g. out of memory, missing configuration or failed dependency).

10 Real World Example



A Kubernetes pod was restarting every few minutes. New Relic showed high memory usage and the logs contained an out of memory error. Increasing the memory limit and fixing a memory leak in the application resolved the issue.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Junior Engineer Practical Lab: Monitor and Troubleshoot an Application

1 Lab Overview



Goal

Set up a Linux environment, deploy a sample application, configure New Relic, generate traffic, and use metrics, logs and traces to troubleshoot a simulated failure.

What You Will Learn

- ✓ Set up and monitor an application
- ✓ Understand infrastructure and application telemetry
- ✓ Write a basic NRQL query
- ✓ Build a dashboard and configure an alert
- ✓ Investigate and fix a real issue
- ✓ Document the full investigation

2 Lab Environment



Linux OS

Ubuntu 24.04 LTS
(or CentOS/Rocky)



Docker

Run a sample application in a container



Sample Application

Simple Node.js app with a few endpoints



New Relic Agent

Infrastructure + APM + logs + container integration

3 Sample Application

Sample Node.js Application (app.js)

JS

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from VERIQTA lab!');
});

app.get('/api/users', (req, res) => {
  res.json({ users: ['Ann', 'DevOps', 'New Relic'] });
});

app.get('/api/error', (req, res) => {
  res.status(500).send('Simulated application error');
});

app.get('/api/slow', async (req, res) => {
  await new Promise(r => setTimeout(r, 3000));
  res.send('Slow response (3s)');
});

app.listen(3000, () => console.log('App running on port 3000'));
```

4 Deploy the Application with Docker

```
# Create project directory
mkdir newrelic-lab && cd newrelic-lab
```

Create Dockerfile

```
cat > Dockerfile <<EOF
FROM node:20
WORKDIR /app
COPY app.js .
RUN npm init -y && npm install express
CMD ["node", "app.js"]
EOF
```

Build and run container

```
docker build -t demo-app .
docker run -d --name demo-app -p 3000:3000 demo-app
```

Test the application

```
curl http://localhost:3000/
curl http://localhost:3000/api/users
```

5 Configure New Relic Monitoring

Install New Relic Infrastructure Agent

```
curl -o newrelic-infra.sh https://download.newrelic.com/
infra_agent/install.sh
sudo bash newrelic-infra.sh
```

Install New Relic APM for Node.js

```
npm install newrelic
```

Add newrelic.js configuration and set NEW_RELIC_LICENSE_KEY.

Enable Logs and Kubernetes (optional)

- ✓ Enable log forwarding for system and application logs.
- ✓ For Kubernetes, install the New Relic Kubernetes integration (if using a cluster).
- ✓ Verify data is flowing in the New Relic UI.

6 Verify Telemetry in New Relic

new relic

- 🔍 Search
- ➕ Add data
- ➕ All capabilities
- 🔗 APM & services
- 🏠 Infrastructure
- 📄 Logs
- 🏠 Kubernetes
- 📊 Dashboards
- 🔔 Alerts & AI
- ⌵ More



APM

Application showing transactions and response time.



Infrastructure

Host metrics (CPU, memory, disk, network).



Logs

Application and system logs visible.



Kubernetes

Cluster, nodes, pods and container metrics (if applicable).

7 Generate Sample Traffic

Use a simple load test with curl

```
for i in {1..1000}; do
  curl -s http://localhost:3000/ > /dev/null
  curl -s http://localhost:3000/api/users > /dev/null
  curl -s http://localhost:3000/api/slow > /dev/null
done
```

8 Inspect Metrics, Logs and Traces



Metrics

Check response time, error rate, throughput and infrastructure metrics.



Logs

Search for errors and correlate timestamps.



Traces

Follow a slow or failed request end-to-end using distributed tracing.

9 Write a Basic NRQL Query

NRQL Examples

Copy

```
SELECT average(responseTime)
FROM Transaction
SINCE 30 minutes ago
TIMESERIES

SELECT count(*)
FROM Transaction
WHERE error IS true
SINCE 1 hour ago
FACET appName
```

10 Build a Monitoring Dashboard

Application Overview



11 Configure a Test Alert

Alert policy

Create a policy (e.g. Application Alerts).



Condition

Set a threshold (e.g. error rate > 5%).



Notification

Send to Slack, email or PagerDuty.



Test

Generate an error and confirm the alert is triggered.

13 Document Your Findings



1. Evidence

Screenshots, logs, metrics and traces.



2. Root Cause

Explain what caused the issue.



3. Corrective Action

Steps taken to fix and prevent recurrence.



4. Verification

Confirm the application is working as expected.

14 Success Criteria

- ✓ Application and infrastructure telemetry visible in New Relic.
- ✓ Able to search logs, view traces and analyze metrics.
- ✓ Dashboard created with key application and system metrics.
- ✓ Test alert configured and triggered.
- ✓ Successfully investigated and resolved a simulated failure.
- ✓ Complete documentation with evidence and root cause.

12 Simulate a Failure and Investigate



Simulate a failure

(e.g. stop the database, introduce high latency or return 500 errors).



Use metrics, logs and traces to identify the root cause.



Apply a fix

(e.g. restart the service or update configuration).



Verify the application is healthy and metrics return to normal.

15 Junior Engineer Tip



When troubleshooting, always correlate metrics, logs and traces. Look at the timeline, compare with recent changes, and follow the evidence step-by-step.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta